

Tech & Teach

Digital Chip Design Program

RTL Design

Almas Alshubrumi

Atheer Al-harbi

Mjd AL-harbi

Maha Omar

Jood Alomair

29/7/2026

Controller – Part 2

Version 1.0

Tech & Teach

Task #14.1

```
module alu #(
    parameter ALU_WIDTH =4
)(
    input logic [ALU_WIDTH-1:0] op1,
    input logic [ALU_WIDTH-1:0] op2,
    input logic [1:0] opcode,
    output logic [ALU_WIDTH-1:0] alu_result
);

always_comb begin
    case(opcode)
        2'b00: alu_result = op1 + op2;
        2'b01: alu_result = op1 - op2;
        2'b10: alu_result = op1 & op2;
        2'b11: alu_result = op1 | op2;
        default: alu_result = 'bx;

    endcase
end

endmodule
```

```
module program_counter # (
    parameter PROG_VALUE = 16
)(
    input logic clk,
    input logic rstn,
    output logic [($clog2(PROG_VALUE))-1:0]
    counter
);

always_ff @(posedge clk or negedge rstn )
begin
    if (!rstn)
        counter <= 0;
    else
        counter <= counter +1;
    end
end

endmodule
```

```
module register_file #( parameter
    REGF_WIDTH = 4)(
    input logic clk,rstn,
    input logic [1:0] rs1,
    input logic [1:0] rs2,
    input logic [1:0] rd,
    input logic [REGF_WIDTH-1:0] alu_result,
    output logic [REGF_WIDTH-1:0] op1,
    output logic [REGF_WIDTH-1:0] op2
);
    logic [REGF_WIDTH-1:0] x [0:3];
    always @ (posedge clk ,negedge rstn)begin
        if(!rstn)
            for(int i = 0;i<REGF_WIDTH;i++)
                x[i]<=0;
            //alu result
            case(rd)
                2'b01: x[1]<= alu_result;
                2'b10: x[2]<= alu_result;
                2'b11 : x[3]<= alu_result;
            default:begin
                x[1]<= x[1];
                x[2]<= x[2];
                x[2]<= (x[2]);
            end
            endcase
        end
        //rs2
        always_comb begin
            case(rs2)
                2'b00:op2=x[0];
                2'b01:op2=x[1];
                2'b10:op2=x[2];
                2'b11:op2=x[3];
            endcase
        end
        //rs1
        always_comb begin
            case(rs1)
                2'b00:op1=x[0];
                2'b01:op1=x[1];
                2'b10:op1=x[2];
                2'b11:op1=x[3];
            endcase
        end
    end
endmodule
```

Task #14.1

```
module instruction_memory #(parameter
IMEM_DEPTH=16,
FILE_NAME="C:\\Users\\alfra\\Desktop\\trin
g\\baby processor\\machin_code.mem")
( input
logic[$clog2(IMEM_DEPTH)-1:0]address,
output logic [7:0] instruction );
logic [7:0] imem [IMEM_DEPTH-1:0];
initial $readmemb(FILE_NAME,imem);
assign instruction=imem[address];
endmodule
```

```
module datapath #(
parameter
IMEM_DEPTH = 16,
PROG_VALUE = 16,
REGF_WIDTH = 4,
ALU_WIDTH = 4,
FILE_NAME =
"C:\\Users\\almas\\Desktop\\test\\Almas\\day17
\\task#14\\machine_code.mem"
)(
input logic clk,
input logic rstn,
input logic [1:0] rs1,
input logic [1:0] rs2,
input logic [1:0] rd,
input logic [1:0] opcode,

output logic [7:0] instruction
);
logic [$clog2(IMEM_DEPTH)-1:0] address;
logic [ALU_WIDTH-1:0] op1;
logic [ALU_WIDTH-1:0] op2;
logic [ALU_WIDTH-1:0] alu_result;
program_counter #(
.PROG_VALUE(PROG_VALUE)
) pc_ins (
.clk(clk),
.rstn(rstn),
.counter(address)
);
```

```
instruction_memory #(
.IMEM_DEPTH(IMEM_DEPTH),
.FILE_NAME(FILE_NAME)
) rom_ins (
.address(address),
.instruction(instruction)
);

register_file #(
.REGF_WIDTH(REGF_WIDTH)
) rf_ins (
.clk(clk),
.rstn(rstn),
.rs1(rs1),
.rs2(rs2),
.rd(rd),
.alu_result(alu_result),
.op1(op1),
.op2(op2)
);
alu #(
.ALU_WIDTH(ALU_WIDTH)
) alu_ins (
.op1(op1),
.op2(op2),
.opcode(opcode),
.alu_result(alu_result)
);
endmodule
```

Task #14.1

```
`timescale 1ns / 1ps
```

```
module top2_tb;
```

```
    logic clk;  
    logic rstn;
```

```
    top2 #(.FILE_NAME("C:\\Users\\JOUDA\\Desktop\\Baby B processor\\machine_code.mem")) t(  
        .clk(clk), .rstn(rstn)  
    );
```

```
    initial begin
```

```
        rstn = 0;  
        clk = 0;  
        #2 rstn = 1;
```

```
    end
```

```
    always #5 clk = ~clk;
```

```
    always @(posedge clk) begin
```

```
        if (rstn)
```

```
            $display("t=%0t | PC=%0d | instr=%b | opcode=%b rs1=%0d rs2=%0d rd=%0d | op1=%0d op2=%0d alu=%0d",  
                $time,  
                t.dpath.pc_ins.counter,  
                t.instruction,  
                t.opcode, t.rs1, t.rs2, t.rd,  
                t.dpath.op1, t.dpath.op2, t.dpath.alu_result);
```

```
    end
```

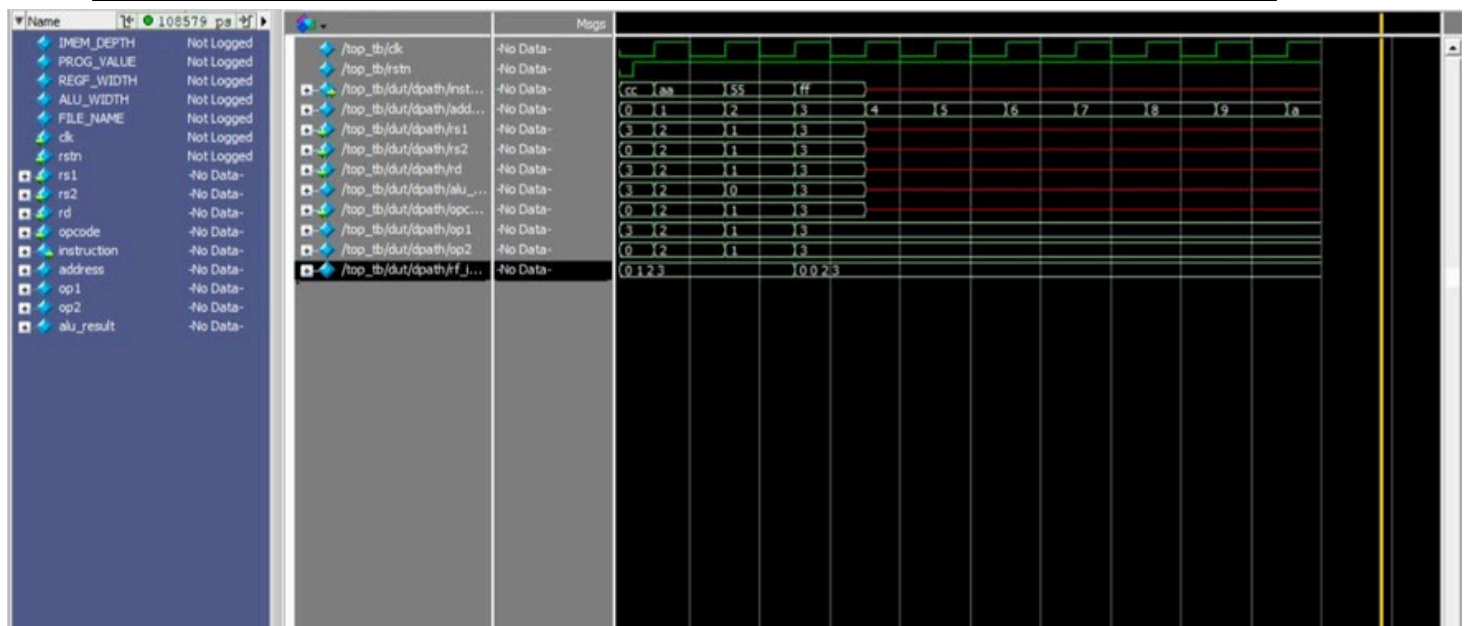
```
    initial begin
```

```
        #100;  
        $display("Final: x1=%0d x2=%0d x3=%0d",  
            t.dpath.rf_ins.x[1],  
            t.dpath.rf_ins.x[2],  
            t.dpath.rf_ins.x[3]);
```

```
        $finish;
```

```
    end
```

```
endmodule
```



Work distribution

Section	Almas	Atheer	Mjd	Maha	Jood
Report	✓	✓	✓	✓	✓
Task13.1	✓	✓	✓	✓	✓
Task13.2	✓	✓	✓	✓	✓
Task13.3	✓	✓	✓	✓	✓

Tech & Teach